

MinneApple: detección, segmentación y atención para estimar la cosecha de manzana

Proyecto 2 · Visión Computacional con Deep Learning Maestría en Inteligencia Artificial y Ciencia de Datos — Universidad Autónoma de Occidente

Integrantes: Alejandro Galvez, Joan Sebastian Mena, Maria Estella Fuentes, Sergio Luis Castaño

Resumen

MinneApple detecta y segmenta manzanas individuales en fotos de huerto con YOLO11s-seg y self-attention (C2PSA): en test obtuvo $mAP@0.5:0.95$ de 0,480 en caja y 0,348 en máscara. La comparación con y sin self-attention, con bootstrap pareado sobre 331 imágenes, confirma que C2PSA mejora la segmentación (Dice +0,017) y el conteo (1,9 manzanas menos de error por foto) — un resultado que en validación (menos datos) no se distinguía del ruido. La cuantización INT8 reduce el modelo un 62 % (39,2 → 14,9 MB) y lo acelera un 20 % en CPU; cumplió la tolerancia de calidad en validación, pero **en test la supera** (caja -0,044): se reporta tal cual. En la GPU (GTX 1050) la red es 7 veces más rápida que en la CPU (120 frente a 846 ms), y la app analiza una foto en 0,3 s.

1. Problemática

Los agricultores necesitan estimar el rendimiento de un cultivo de manzanas antes de la cosecha, para planificar la logística de recolección, almacenamiento y distribución. Hacerlo a ojo es impreciso, y ese error se traduce en desperdicio de fruta o en falta de capacidad de transporte y almacenamiento contratada de más o de menos.

El sistema resuelve tres tareas sobre la misma foto de un árbol:

- **Detección:** contar cada manzana visible mediante una caja, para estimar el volumen del huerto.
- **Segmentación de instancias:** separar cada manzana de las hojas, ramas y del fondo, para medir su diámetro real.
- **Atención:** un mecanismo de self-attention integrado en el modelo (C2PSA) y una herramienta de interpretabilidad (Grad-CAM a mano) sobre las predicciones.

El problema es difícil por tres razones, verificadas en los datos:

- **Objetos muy pequeños.** Entre el 75 % y el 89 % de las manzanas mide menos de 32×32 px en una imagen de 1280×720; el lado mediano es de 30-35 px (`reports/eda.json`).
- **Oclusión y anotación incompleta.** Las manzanas se tapan entre sí y con las hojas; además, el dataset **no anota** las manzanas caídas al suelo ni las de árboles del fondo (verificado por inspección visual, 26-sep-2026), lo que pone un techo a cualquier métrica.
- **Variación de iluminación y color.** Fotos tomadas en 2015-2016 con el lado soleado y el sombreado de las hileras; la mezcla de manzanas rojas y verde/amarillo cambia entre entrenamiento (56,6 %) y test (35,5 %), según un descriptor propio de color (fracción de píxeles rojos de la máscara, sin validar contra ninguna etiqueta).

2. Dataset

MinneApple (Häni, Roy, Isler, 2019), 670 imágenes de entrenamiento con máscaras de instancia y 331 de test oficial con etiquetas (publicadas en 2022). Licencia **CC BY-NC-SA 3.0 US**. Detalle completo, comandos de descarga y verificación de MD5 en el README del repositorio.

Partición: train se divide en train/val **por video completo** (dos videos para val, 93 imágenes), porque los fotogramas de un mismo video son casi idénticos y partir por imagen inflaría las métricas (se verificó por código que ningún video cae en dos particiones). El test oficial (331 imágenes, otras hileras, otro año) se usa una sola vez, al final.

3. Decisiones de arquitectura

Decisión	Elección	Por qué	Alternativa descartada
Detección + segmentación	Un solo modelo , YOLO11s-seg	Separa manzanas que se tocan (necesario para el diámetro); un solo entrenamiento cabe en el plazo	YOLO-det + U-Net separada (dos entrenamientos; la U-Net semántica no separa instancias)
Self-attention	C2PSA (capa 10 del backbone, incluida en YOLO11)	Ya viene en la arquitectura; se compara con una variante idéntica donde esa capa se reemplaza por nn.Identity	Añadir un bloque de atención propio (reinventar lo que ya trae el modelo)
Interpretabilidad	Grad-CAM a mano , sobre la capa 16 (paso 8)	Es la única de las 3 capas candidatas cuyo mapa no sale vacío con manzanas de este tamaño (cociente de enfoque 54,0 frente a 47,9 y 0,9)	Capas 19/22 (peor enfoque, mapas vacíos); librerías externas de CAM
Madurez / color	Descriptivo, no evaluado	El dataset no trae etiquetas de madurez ni de manzanas podridas	Anotar madurez a mano (no cabe en el plazo)
Hiperparámetros de entrenamiento	Lote 4, mask_ratio=8, 40 épocas, sin parada temprana; 1024 px en local (ronda 1 y ablación) y 1280 px en Colab (modelo final)	La resolución nativa (1280 px) no cabe en 4 GB de VRAM; sin parada temprana para que las 6 corridas de la ablación sean directamente comparables entre sí	Entrenar a 1280 px local (se probó: 5,77 GiB, no cabe); parada temprana
Mejora tras la ronda 1	Small a 1280 px (Colab, por falta de VRAM local a esa resolución); +0,020 de mAP de caja sobre nano, confirmado con 2 semillas; 332 ms por foto en la GPU local	El recall de manzanas < 20 px era del 50 % en la ronda 1; scale reducido (T2) no mejoró (dentro del ruido)	Más épocas, entrenar a 1280 con recortes (se descartaron por tiempo)

4. Proceso de optimización

Protocolo fijado antes de medir:

- Exportación a **ONNX** (forma fija 1280×1280 para el modelo final, para que Ultralytics pueda calcular el mAP del modelo exportado).
- Cuantización **INT8 estática** con ONNX Runtime (formato QDQ, por canal, calibrada con 100 imágenes reales de entrenamiento).
- **Criterio de aceptación:** en validación, con el mismo motor, la caída de mAP@0.5:0.95 de caja y de máscara no supera **0,02**. Si no se cumple, se prueba (en orden) excluir la cabeza de la cuantización, o FP16; si nada pasa, se reporta tal cual.
- Comparación de tamaño: el .pt de Ultralytics está en FP16 (19,6 MB), así que la comparación real de la optimización es **ONNX FP32 frente a ONNX INT8**.

Intento A (todo en INT8): mAP 0. La salida del modelo mezcla coordenadas (0 a 1.280 px) y confianzas (0 a 1) en un mismo tensor; con una sola escala de 8 bits cada nivel vale unos 5 px y todas

las confianzas quedan en 0. **Intento B (cabeza en FP32): aceptado en validación.** Después se evaluó en test una sola vez, solo para reportar:

Métrica	FP32 (ONNX)	INT8 (ONNX, intento B)	Diferencia en validación	Diferencia en test
Tamaño (MB)	39,16	14,93	−61,9 %	—
mAP@0.5:0.95 caja (val / test)	0,4612 / 0,4821	0,4447 / 0,4379	−0,0165 (cumple)	−0,0442 (no cumple)
mAP@0.5:0.95 máscara (val / test)	0,3643 / 0,3472	0,3532 / 0,3223	−0,0112 (cumple)	−0,0249 (no cumple)
Manzanas predichas de más por foto (bootstrap pareado)	—	—	+2,5	+9,5 (IC95 8,5 a 10,5)

El INT8 generaliza peor que el FP32: en test «inventa» manzanas (precisión de caja 0,74 frente a 0,82), sobre todo en los conjuntos `dataset1_back` y `dataset2_back`. Explicación probable, no verificada: las 100 fotos de calibración salen de los mismos videos que validación, y el test oficial es de otras hileras y otro año. No se prueba otra variante mirando el test; la app usa el `.pt` (necesita gradientes para Grad-CAM, que el ONNX no permite), así que no se afecta (`scripts/comparar_int8.py`).

5. Tiempos de ejecución: CPU frente a GPU

Protocolo: calentamiento, 50 mediciones por caso (mediana y p95), `cuda.synchronize()` en GPU, casos intercalados en orden aleatorio, hardware exacto por fila, una sola corrida por método. Se mide el **forward** del modelo solo y el **pipeline** completo (preproceso + modelo + NMS + máscaras), y lo que hace la **app** con una foto. Hardware: CPU Intel Core i5-8300H @ 2,30 GHz (4 núcleos; se miden 4 hilos), GPU NVIDIA GeForce GTX 1050 (4 GB); PyTorch 2.6.0+cu124, ONNX Runtime 1.30.0. El script (`scripts/benchmark.py`) se niega a medir si detecta un entrenamiento en marcha, para no falsear las cifras. Datos: `reports/benchmark.csv` (27-sep, modelo final, 1280 px, confianza 0,35).

Motor	Dispositivo	Etapas (entrada)	Mediana (ms)	p95 (ms)
PyTorch (.pt)	CPU	forward (1280×1280)	846	1.489
PyTorch (.pt)	GPU	forward (1280×1280)	120 (7,0× más rápida)	214
ONNX Runtime FP32	CPU	forward (1280×1280)	813	2.285
ONNX Runtime INT8	CPU	forward (1280×1280)	659 (1,23× sobre FP32)	1.178
PyTorch (.pt)	CPU	pipeline completo (1280×736)	480	561
PyTorch (.pt)	GPU	pipeline completo (1280×736)	101 (4,8×)	148
ONNX Runtime FP32	CPU	pipeline completo (1280×1280)	993	1.076
ONNX Runtime INT8	CPU	pipeline completo (1280×1280)	826 (1,20× sobre FP32)	901
App (.pt)	CPU / GPU	análisis de una foto	990 / 323	1.175 / 371
App (.pt) con Grad-CAM	CPU / GPU	análisis + mapa	1.530 / 923	3.011 / 1.047

Lectura: la GPU acelera 7 veces la red y 3 veces la app completa (el resto es preproceso, máscaras y medidas, que corren en CPU). El INT8 gana poco en velocidad (20 %): la cabeza sigue en FP32 y el i5-8300H no tiene instrucciones VNNI para enteros de 8 bits (razones probables, no perfiladas). El pipeline ONNX es más lento que el del `.pt` porque procesa un cuadrado con 1,74 veces más píxeles. Algunos p95 de CPU son altos por la variación de frecuencia del portátil: la mediana es la cifra de

referencia.

6. Resultados

Modelo final (`colab_1280_small_s0`, YOLO11s-seg + C2PSA, 1280 px), test (331 imágenes), evaluado una sola vez:

Métrica	Valor
mAP@0.5:0.95 caja	0,480 (mAP@0.5: 0,819)
mAP@0.5:0.95 máscara	0,348 (mAP@0.5: 0,757)
Dice de la unión / IoU	0,782 / 0,648
Error de conteo por foto (MAE)	5,4 manzanas (sesgo -2,8, subcuenta ligera)

Referencia orientativa del artículo original (Faster R-CNN, otro test): mAP de caja 0,438. El modelo final queda por encima.

7. Atención: qué aporta cada mecanismo

Self-attention (C2PSA). Comparación con y sin C2PSA, 3 semillas por variante, bootstrap pareado por imagen en test (`scripts/comparar_ablacion.py`, 993 pares). **Se confirma una mejora** en Dice de la unión (+0,017, IC95 0,014 a 0,019) y en error de conteo (1,9 manzanas menos de error por foto, IC95 -2,29 a -1,53); en mAP de caja la mejora es más modesta y no se sometió a bootstrap (+0,006, media de 3 semillas). El dry run exploratorio en **validación** (93 imágenes) no había mostrado ninguna diferencia distinguible del ruido (`reports/ablacion_val_exploratorio.json`) — con más datos de evaluación (test tiene 331 imágenes) el efecto se volvió medible. Es un hallazgo honesto: no es que el bloque «empezara a funcionar», es que antes no había evidencia suficiente para verlo.

Grad-CAM. Sobre la capa 16, la atención se concentra fuertemente en las manzanas detectadas: en una imagen de ejemplo, el 48 % de la energía del mapa cae sobre el 2,4 % de la imagen que ocupan (20 veces lo esperado al azar); en el barrido de 20 imágenes de validación que decidió la capa, el cociente medio fue de 54,0.

8. Limitaciones y alcances

Ver la model card (`MODEL_CARD.md`), secciones 5 y 6, para el detalle completo (manzanas diminutas, anotación incompleta, un solo huerto, sin etiquetas de madurez, diámetro y peso estimados).

9. Conclusiones

1. **Un solo modelo resuelve las tres tareas.** YOLO11s-seg a 1280 px detecta (mAP de caja 0,480 en test, por encima del 0,438 de referencia del artículo, en otro test) y segmenta cada manzana (mAP de máscara 0,348; Dice 0,78), lo que permite aislarla y medirla.
2. **La self-attention aporta, pero poco, y hizo falta el test para verlo.** Con C2PSA el Dice sube 0,017 y el error de conteo baja 1,9 manzanas por foto (IC95 sin cruzar el cero, 993 pares). En validación (93 imágenes) el efecto no se distinguía del ruido.
3. **Lo que más mejoró el modelo fue la capacidad y la resolución**, no la atención: pasar de nano a small y entrenar a 1280 px sumó +0,020 de mAP de caja en validación. El límite sigue siendo el de siempre: manzanas diminutas y oclusión.
4. **La optimización hay que validarla fuera de la distribución de calibración.** El INT8 cumplió la tolerancia en validación y la incumplió en test (caja $-0,044$, conteo +9,5 manzanas por foto). La compresión da un 62 % menos de tamaño pero solo un 20 % más de velocidad en esta CPU; para la app se mantiene el .pt.
5. **La GPU marca la diferencia en la experiencia de uso:** 0,3 s por foto frente a 1 s en CPU (0,9 s frente a 1,5 s con Grad-CAM).
6. **Trazabilidad:** cada cifra de este informe sale de `reports/`; el umbral de confianza, la tolerancia del INT8 y el criterio de la ablación se fijaron **antes** de ver el resultado, y el test se evaluó una sola vez.

10. Posibles mejoras (trabajo futuro)

Cambiar la arquitectura base a YOLO26. Se entrenó, de forma exploratoria, YOLO26n-seg (nano) sobre el mismo dataset y la misma partición, con y sin su bloque de atención (C2PSA, el mismo que usa YOLO11). Evaluado con nuestro propio protocolo (mismo val, `mask_ratio=1`, umbral de confianza propio buscado igual que el nuestro):

	Nuestro final (small + C2PSA)	YOLO26n sin atención	YOLO26n con atención
Parámetros / GFLOPs	10,08 M / 33,2	2,80 M / 10,0	3,05 M / 10,3
Mask mAP50-95	0,364	0,377	0,373
Dice / IoU	0,791 / 0,658	0,801 / 0,670	0,807 / 0,679
Error de conteo (sesgo)	4,40 (−3,73)	3,30 (−0,18)	3,55 (+0,32)

Con **un tercio de los parámetros**, YOLO26n iguala o supera a nuestro modelo final en Dice, IoU y sesgo de conteo. La atención aporta poco en YOLO26 (Dice +0,006), del mismo orden que en nuestro propio resultado — consistente con que el techo actual es la arquitectura base, no el bloque de atención. **No se adoptó para esta entrega:** solo se evaluó en validación (nunca en test, el caso del INT8 de este mismo informe es la advertencia de por qué eso importa), con una sola semilla y sin el resto del protocolo (test una sola vez, INT8, tiempos, notebook). Queda como la mejora más prometedora para una siguiente iteración del proyecto.

Otras líneas abiertas: recalibrar la cuantización INT8 con imágenes más variadas (en test perdió más precisión de la tolerada, sección 4); investigar por qué el modelo detecta peor las manzanas verde/amarillo que las rojas (0,61 frente a 0,86 de recall en validación, sin explicarse por tamaño — ver la model card, limitaciones); repetir la ablación de C2PSA con el modelo final (small a 1280 px) en vez de con el nano de 1024 px.